Smart Plant Monitoring and Automated Irrigation System

Individual Practical Assignment

By Julian Hess (105922390)

1. Summary

Current agriculture practices and home gardening face a constant challenge of ensuring plants receive enough water without constant human supervision. Overwatering wastes resources, while underwatering can damage or even kill plants. The Smart Plant Monitoring and Automated Irrigation System addresses this problem by combining low-cost hardware with edge computing to provide real-time environmental monitoring and fully automated watering.

This project implements a complete IoT architecture consisting of four layers, physical sensing and actuation, serial communication, edge computing and database, and a web-based user interface layer for monitoring and analysis. An Arduino Uno collects environmental data from two sensors, an analog soil moisture sensor, and a digital DHT22 temperature and humidity sensor. It then transmits this data in real time to an edge computing device (a Mac running Python scripts), which stores readings in a MariaDB database, applies conditional automation rules and serves a Flask-based monitoring dashboard.

When soil moisture drops below a threshold pre-determined as “dry”, the edge server instructs the Arduino to activate a 5V relay, which switches on a submersible water pump connected to an external power supply. A status LED provides immediate visual confirmation of actuation. This design ensures plants are watered precisely when needed, while the web dashboard gives users the ability to monitor real-time data and to perform analysis.

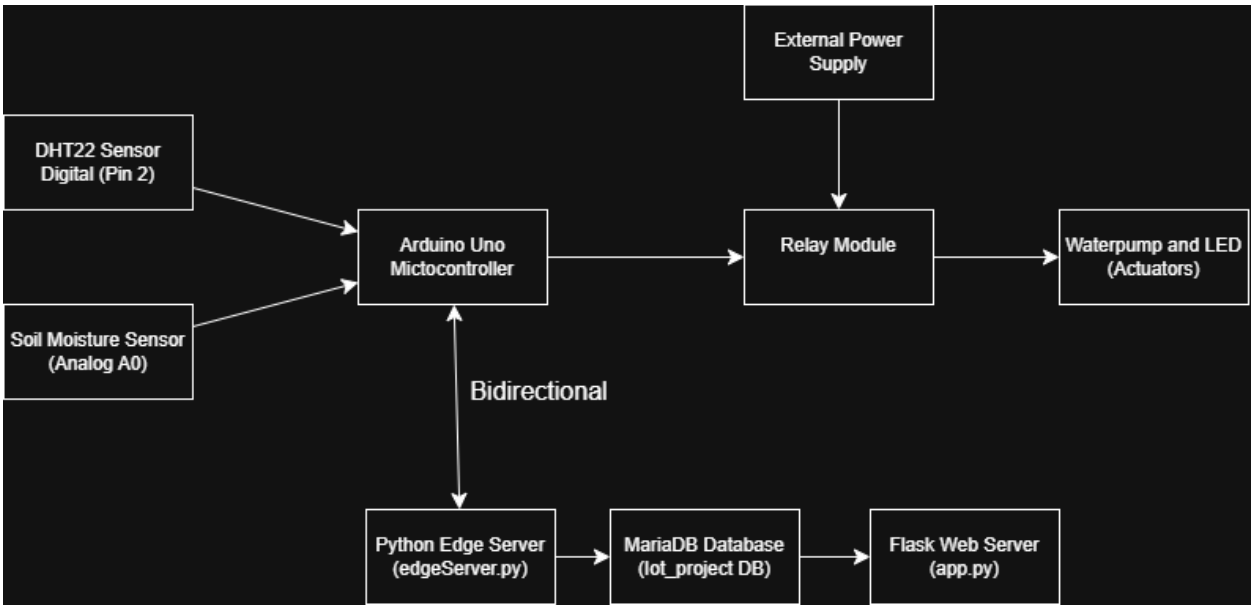
1.1 System Overview

Component	Description
Physical Layer	Arduino Uno with analog soil moisture sensor, digital DHT22 temperature and humidity sensor, relay module, water pump powered by external power supply and status LED
Communication	USB Serial (9600 baud) bidirectional CSV data from Arduino, command strings from edge server to Arduino
Edge Server	Python (pyserial, mysql-connector-python) running on Mac, data ingestion, analytics, automation rules.
Database	MariaDB – iot_project database with sensor_data table storing timestamped data readings
User Interface	Flask web application with live dashboard (auto-refresh every 2 s) and analytics buttons (Avg / Min / Max)

2. Conceptual Design

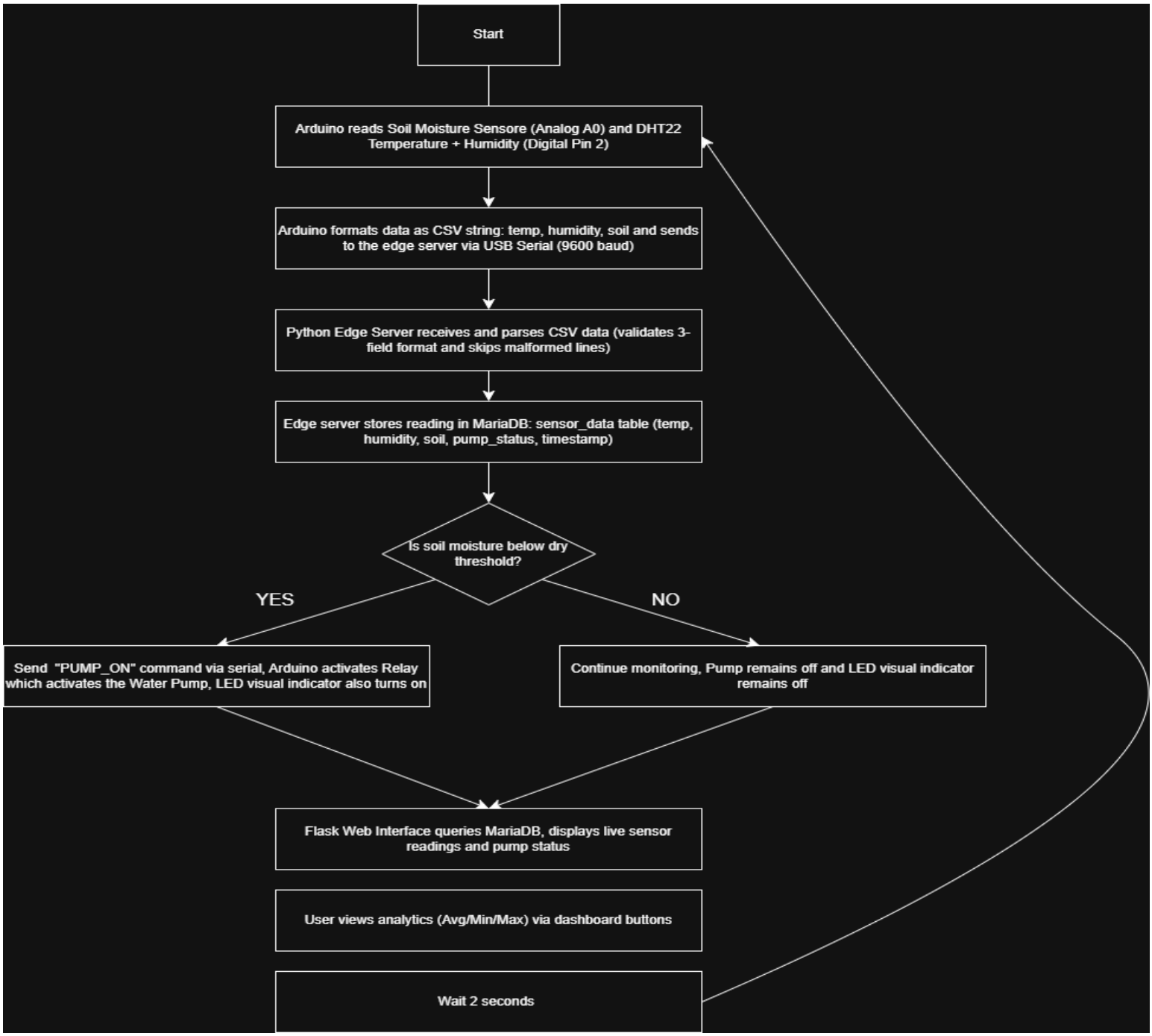
2.1 Block Diagram

The block diagram below shows the hardware and software components across all layers of the IoT architecture and how data flows between them.



2.2 Activity Flowchart

The flowchart below describes the complete software flow, from initial sensor reading through to user interface interaction, including the conditional pump activation decision.



3. Implementation

3.1 Sensors

3.1.1 Analog Sensor – Soil Moisture Sensor

What's being measured: The water content of the plant's soil, expressed as a raw integer value in the range of 0-1023 (Arduino 10-bit ADC resolution).

Why measure it? Soil moisture is the primary trigger for irrigation systems. Too little moisture stresses the plant, but too much moisture causes the root to rot. By measuring moisture continuously, the system can water only when it's actually needed which conserves water and protect the plants' health.

Integration with Arduino: The sensor module outputs an analog voltage which is proportional to soil conductivity. To improve accuracy, the Arduino takes 10 successive analogRead(A0) readings with a 10ms delay between each and returns the average result. This smoothing reduces noise from electrical interference which I was receiving from the external power supply and gives more stable soil moisture values. Lower ADC values correspond to drier soil (higher resistance), and higher values to wetter soil (lower resistance). A configurable threshold (dry threshold) determines when the pump should be automatically activated.

Parameter	Detail
Sensor type	Analog resistive
Arduino pin	A0 (Analog input)
Power	5V, GND
Output range	0-1023 (10-bit ADC)
Dry threshold	250 or below, the soil is considered dry, pump activates

3.1.2 Digital Sensor – DHT22 Temperature and Humidity Sensor

What is being measured: Ambient air temperature (degrees Celsius) and humidity (percentage).

Why measure it? High temperature and low humidity both accelerate soil moisture evaporation and increase plant water demand. These readings provide additional environmental context and allow for secondary automation rules, such as issuing a high-temperature warning when the edge server detects readings above 30 degrees.

Integration with Arduino: The DHT22 uses a single-wire digital protocol. The DHT library handles timing and decoding, so readings are obtained with dht.readTemperature() and dht.readHumidity(). Invalid readings (caused by electrical noise or sensor timeout) are filtered using isnan() before being included in the serial output string. This prevents corrupt data from reaching the edge server.

Parameter	Detail
Sensor type	Digital single-wire (DHT protocol)
Arduino pin	Digital Pin 2
Temperature range	Capable of –40 Celsius to 80 Celsius give or take 0.5 degrees
Humidity range	Capable of 0-100% with the low humidity threshold set at 30% and below
Library	DHT sensor library by Adafruit

3.2 Actuators

3.2.1 Actuator 1 – Water Pump (via Relay Module)

What it does: The 5V submersible mini water pump draws water from a water supply (in my case it was a bowl of water) and delivers it to the plant soil (in my design it just sent water to another empty bowl so that water did not spill over the electronics while testing). Because the pump's current draw exceeds what an Arduino digital output pin can safely supply, a 5V single-channel relay module is used as an electrically isolated switch between the Arduino and the pump's external power supply.

How it's integrated: The pump is controlled through two mechanisms. First, if soil moisture falls below 250 (dry threshold) or humidity drops below 30%, the Arduino immediately sets Pin 7 HIGH to activate the relay module and pump and Pin 8 HIGH to activate the LED. A built-in timeout of 3 seconds automatically shuts the pump off after each watering cycle which prevents overwatering regardless of edge server state. The pump also stops when soil rises above 400 (wet threshold) and humidity recovers above 35%. Second, the Python edge server monitors the same readings and sends "PUMP_ON" / "PUMP_OFF" commands via serial as an additional control layer.

Parameter	Detail
Pump type	5V DC submersible mini pump
Relay module	5V single-channel relay, controlled via Pin 7
External supply	Separate 5V power adapter (pump circuit isolated from Arduino)

Trigger command	“PUMP_ON” / “PUMP_OFF” received via serial from edge server

3.2.2 Actuator 2 – Status LED

What it does: A standard 5mm LED provides an immediate visual indicator of the pump’s current state (on or off). When the pump is active, the LED illuminates. When the pump is off, the LED is turned off. This allows an observer to confirm actuation immediately without needing to access the web interface.

How it is integrated: The LED is connected through a resistor to Arduino Digital Pin 8. The LED state is updated simultaneously with the relay state, so both actuators always work in tandem.

Parameter	Detail
Component	Standard 5mm LED (green)
Arduino Pin	Digital Pin 8
Resistor	220 ohm current-limiting resistor
Behavior	ON when pump active and OFF when pump inactive

3.3 Software and Libraries

3.3.1 Arduino Sketch

The Arduino sketch (plant_monitor.ino) is responsible for all physical-layer operations. It initializes the DHT22 using the Adafruit DHT library, reads both sensors on a fixed loop interval, constructs a CSV string in the format of temperature, humidity, soil_moisture and transmits this over Serial at 9600 baud. The sketch also listens for incoming serial commands. Upon receiving “PUMP_ON”, it activates both the relay (Pin 7) and the LED (Pin 8). Upon receiving “PUMP_OFF”, it deactivates both. A key design feature is that the Arduino can activate the pump based on dry soil or low humidity readings without waiting for a Python command. This ensures the plant is protected even if the edge server goes down.

3.3.2 Python Edge Server (edgeServer.py)

The edge server script runs inside a Python virtual environment on the Mac terminal. It uses the pyserial library to open the correct serial port (/dev/tty.usbserial-A5069RR4) at 9600 baud and reads lines from the Arduino in a continuous loop.

Validated readings are unpacked into temperature, humidity, and soil_moisture variables. The edge server then applies the conditional automation rule, if soil_moisture < 250, a "PUMP_ON" command is written back to the serial port. If the reading is above the wet threshold, "PUMP_OFF" is sent. Temperature is also checked, if readings are above 30 degrees Celsius then a warning message is produced.

All readings are sent to MariaDB via mysql-connector-python. Analytics (average, minimum, maximum soil moisture) are computed using SQL aggregate functions and made available to view on the Flask web application.

3.3.3 Flask Web Interface (app.py)

The Flask application runs on localhost and serves the monitoring dashboard. It connects to MariaDB to retrieve the most recent sensor reading for live display, and exposes three analytics endpoints that execute AVG(), MIN(), and MAX() queries against the sensor_data table. The HTML template includes <meta http-equiv="refresh" content="2"> so that the dashboard automatically reloads every two seconds. Users can trigger the different analytics by clicking the button for average, minimum, or maximum, which all call the respective Flask routes and display the computed result.

3.3.4 Libraries Used

Library	Language	Purpose
DHT (Adafruit)	C++ (Arduino)	Reads temperature and humidity from DHT22 sensor
pyserial	Python	Opens and reads/writes the USB serial port between Mac and Arduino
Mysql-connector-python	Python	Connects to MariaDB and executes INSERT and SELECT queries
Flask	Python	Lightweight web framework serving the monitoring dashboard

5. Resources

These are the list of resources consulted during the design and development of the system.

1. Arduino analogRead()
   https://www.arduino.cc/reference/en/language/functions/analog-io/analogread/
2. Arduino Serial Communication
   https://www.arduino.cc/reference/en/language/functions/communication/serial/
3. Adafruit DHT Sensor Library (GitHub)
   https://github.com/adafruit/DHT-sensor-library
4. Adafruit DHT22 Tutorial
   https://learn.adafruit.com/dht
5. PySerial Documentation
   https://pyserial.readthedocs.io/en/latest/
6. Flask Documentation
   https://flask.palletsprojects.com/en/latest/quickstart/
7. MariaDB Documentation
   https://mariadb.com/kb/en/documentation/
8. MySQL Connector/Python Developer Guide
   https://dev.mysql.com/doc/connector-python/en/

6. Appendix – Source Code

Appendix A – Arduino Sketch (plant_monitor.ino)

```
#include <DHT.h>

#define DHTPIN 2
#define DHTTYPE DHT22

const int soilPin = A0;
const int relayPin = 7;
const int ledPin = 8;

DHT dht(DHTPIN, DHT22);

bool pumpOn = false;
unsigned long pumpStartTime = 0;
const unsigned long pumpRunTime = 3000;

//Thresholds
const int dryThreshold = 250;
const int wetThreshold = 400;
const float lowHumidity = 30.0;

//soil reading smoothing to deal with browning out arduino
int readSoil() {
long sum = 0;

for (int i = 0; i < 10; i++) {
sum += analogRead(soilPin);
delay(10);
}

return sum / 10;
}

//Humidity
float readHumidity() {
float h = dht.readHumidity();
if (isnan(h)) return -1;
return h;
}

//Temp
float readTemp() {
float t = dht.readTemperature();
if (isnan(t)) return -1;
return t;
}

void setup() {
Serial.begin(9600);
```

```
pinMode(relayPin, OUTPUT);
pinMode(ledPin, OUTPUT);

digitalWrite(relayPin, LOW); //turn pump off
digitalWrite(ledPin, LOW); //turn LED off

dht.begin();

Serial.println("System Starting...");
}

void loop() {

int soil = readSoil();
float humidity = readHumidity();
float temp = readTemp();

//serial output for python script
Serial.print(temp);
Serial.print(",");
Serial.print(humidity);
Serial.print(",");
Serial.println(soil);

//rules for safety control
bool drySoil = soil < dryThreshold;
bool wetSoil = soil > wetThreshold;
bool lowHum = (humidity > 0 && humidity < lowHumidity);

//condition for pump on
if ((drySoil || lowHum) && !pumpOn) {

digitalWrite(relayPin, HIGH);
digitalWrite(ledPin, HIGH);

pumpOn = true;
pumpStartTime = millis();

Serial.println("Pump ON");
}

//condition for pump off
if (pumpOn &&
(millis() - pumpStartTime > pumpRunTime ||
(wetSoil && humidity > lowHumidity + 5))) {

digitalWrite(relayPin, LOW);
digitalWrite(ledPin, LOW);
```

```
pumpOn = false;

Serial.println("Pump OFF");
}

delay(500);
}
```

Appendix B – Python Edge Server (edgeServer.py)

```
import serial
import mysql.connector
import time


#serial setup
ser = serial.Serial('/dev/tty.usbserial-A5069RR4', 9600)
time.sleep(2)



#database setup
db = mysql.connector.connect(
host="localhost",
user="root",
password="",
database="iot_project"
)

cursor = db.cursor()

#variables for analytics
total_soil = 0
count = 0

#main
while True:
try:
#flush buffer so we read the LATEST data, not stale buffered lines
ser.reset_input_buffer()
line = ser.readline().decode().strip()

if not line:
continue

print("Received:", line)
```

```
if line.count(",") != 2:
print("Skipping:", line)
continue

temp, hum, soil = line.split(",")

temp = float(temp)
hum = float(hum)
soil = int(soil)

print(f"Temp: {temp}°C | Humidity: {hum}% | Soil: {soil}")

if temp > 30:
print("Warning: High Temperature Detected")

total_soil += soil
count += 1
avg_soil = total_soil / count
print(f"Avg Soil Moisture: {avg_soil:.1f}")

if soil < 250:
pump_status = "ON"
ser.write(b"PUMP_ON\n")
else:
pump_status = "OFF"
ser.write(b"PUMP_OFF\n")

print(f"Pump: {pump_status}")

sql = """
INSERT INTO sensor_data (temperature, humidity, soil_moisture, pump_status)
VALUES (%s, %s, %s, %s)
"""
cursor.execute(sql, (temp, hum, soil, pump_status))
db.commit()

print("Saved to DB\n")

time.sleep(2)

except ValueError as e:
print(f"Parse error on line '{line}': {e}")
except mysql.connector.Error as e:
print(f"DB error: {e}")
except Exception as e:
print(f"Unexpected error: {e}")
```

Appendix C – Flask Web Interface (app.py)

```
from flask import Flask, render_template, request
import mysql.connector

app = Flask(__name__)

def get_db_connection():
return mysql.connector.connect(
host="localhost",
user="root",
password="",
database="iot_project"
)

@app.route("/")
def index():

db = get_db_connection()
cursor = db.cursor()

#latest sensor data
cursor.execute("""
SELECT temperature, humidity, soil_moisture, pump_status
FROM sensor_data
ORDER BY id DESC
LIMIT 1
""")

latest = cursor.fetchone()

result = ""

analysis = request.args.get("analysis")

#for analytics
if analysis == "avg":

cursor.execute("""
SELECT AVG(soil_moisture)
FROM sensor_data
""")

avg = cursor.fetchone()[0]

result = f"Average Soil Moisture: {avg}"

elif analysis == "min":
```

```
cursor.execute("""
SELECT MIN(soil_moisture)
FROM sensor_data
""")

minimum = cursor.fetchone()[0]

result = f"Minimum Soil Moisture: {minimum}"

elif analysis == "max":

cursor.execute("""
SELECT MAX(soil_moisture)
FROM sensor_data
""")

maximum = cursor.fetchone()[0]

result = f"Maximum Soil Moisture: {maximum}"

db.close()

return render_template(
"index.html",
data=latest,
result=result
)

if __name__ == "__main__":
app.run(debug=True)
```

Appendix D – HTML Dashboard Template (index.html)

```
<!DOCTYPE html>
<html>

<head>
<title>IoT Dashboard</title>
<meta http-equiv="refresh" content="2">
</head>

<body>

<h1>IoT Sensor Dashboard</h1>

{% if data %}

<p>Temperature: {{ data[0] }} °C</p>
```

```
<p>Humidity: {{ data[1] }} %</p>

<p>Soil Moisture: {{ data[2] }}</p>

<p>Pump Status: {{ data[3] }}</p>

{% endif %}

<hr>

<h2>Soil Moisture Analysis</h2>

<form method="GET">

<button type="submit" name="analysis" value="avg">
Show Average
</button>

<button type="submit" name="analysis" value="min">
Show Minimum
</button>

<button type="submit" name="analysis" value="max">
Show Maximum
</button>

</form>

{% if result %}

<p>{{ result }}</p>

{% endif %}

</body>
</html>
```

Appendix E – Database Setup SQL

```
CREATE DATABASE iot_project;

USE iot_project;

CREATE TABLE sensor_data (
  id INT AUTO_INCREMENT PRIMARY KEY,
  temperature FLOAT,
  humidity FLOAT,
  soil_moisture INT,
  pump_status VARCHAR(10),
  timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```